

EXPERIMENT 4

EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

Aim :

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

Algorithm :

1. Loading and Viewing Data
2. Finding Shape of Dataset
3. Checking Data Types
4. Filtering Data
5. Sorting Data
6. Statistical Analysis
7. Descriptive Statistics
8. Grouping and Aggregation
9. Correlation Analysis
10. Bar Chart
11. Scatter Plot
12. Histogram

PROGRAM :

```
#import pandas
import pandas as pd

#load Instagram_Analytics.csv file
df = pd.read_csv("../Instagram_Analytics.csv")

===== FIRST FIVE RECORDS =====

#print first five records
print("===== FIRST FIVE RECORDS =====")
df.head()
```

```
===== FIRST FIVE RECORDS =====
   post_id  account_id  account_type  follower_count  media_type  content_category  traffic_source  has_call_to_action  post_datetime  post_date  ...  comi
0  IG0000001         7         brand         3551         reel         Technology         Home Feed             1      2024-11-30 06:00:00  2024-11-30  ...
1  IG0000002        20         creator        31095         image          Fitness         Hashtags             1      2025-08-15 15:00:00  2025-08-15  ...
2  IG0000003        15         brand          8167         reel          Beauty         Reels Feed             0      2025-09-11 16:00:00  2025-09-11  ...
3  IG0000004        11         creator          9044         carousel          Music         External             0      2025-09-18 03:00:00  2025-09-18  ...
4  IG0000005         8         creator        15986         reel         Technology         Profile             0      2025-03-21 09:00:00  2025-03-21  ...
```

5 rows x 23 columns

```
#import pandas
import pandas as pd

#load Instagram_Analytics.csv file
df = pd.read_csv("../Instagram_Analytics.csv")

===== FIRST FIVE RECORDS =====

#print first five records
print("===== FIRST FIVE RECORDS =====")
df.head()

===== FIRST FIVE RECORDS =====
```

	post_id	account_id	account_type	follower_count	media_type	content_category	traffic_source	has_call_to_action	post_datetime	post_date	...	comi
0	IG0000001	7	brand	3551	reel	Technology	Home Feed	1	2024-11-30 06:00:00	2024-11-30	...	
1	IG0000002	20	creator	31095	image	Fitness	Hashtags	1	2025-08-15 15:00:00	2025-08-15	...	
2	IG0000003	15	brand	8167	reel	Beauty	Reels Feed	0	2025-09-11 16:00:00	2025-09-11	...	
3	IG0000004	11	creator	9044	carousel	Music	External	0	2025-09-18 03:00:00	2025-09-18	...	
4	IG0000005	8	creator	15986	reel	Technology	Profile	0	2025-03-21 09:00:00	2025-03-21	...	

5 rows x 23 columns

```
#filter follower count having more than 1000 followers
print("\n===== POSTS WITH FOLLOWER COUNT > 10000 =====")
display(df[df["follower_count"] > 10000].head())

===== POSTS WITH FOLLOWER COUNT > 10000 =====
```

	post_id	account_id	account_type	follower_count	media_type	content_category	traffic_source
1	IG0000002	20	creator	31095	image	Fitness	Hashtags
4	IG0000005	8	creator	15986	reel	Technology	Profile
10	IG0000011	8	creator	15986	reel	Fitness	Home Feed
11	IG0000012	3	creator	10018	carousel	Beauty	Hashtags
12	IG0000013	2	brand	22291	image	Beauty	Explore

5 rows x 23 columns

```

: #display account type who are creators
print("\n===== CREATOR ACCOUNTS =====")
display(df[df["account_type"] == "creator"].head())

```

===== CREATOR ACCOUNTS =====

	post_id	account_id	account_type	follower_count	media_type	content_category	traffic_source
1	IG0000002	20	creator	31095	image	Fitness	Hashtags
3	IG0000004	11	creator	9044	carousel	Music	External
4	IG0000005	8	creator	15986	reel	Technology	Profile
6	IG0000007	19	creator	3447	carousel	Food	Explore
9	IG0000010	4	creator	4972	reel	Music	Hashtags

5 rows × 23 columns

```

#print accounts which has performance_bucket_label are viral
print("\n===== VIRAL POSTS =====")
display(df[df["performance_bucket_label"] == "viral"].head())

```

===== VIRAL POSTS =====

	post_id	account_id	account_type	follower_count	media_type	content_category
1	IG0000002	20	creator	31095	image	Fitness
6	IG0000007	19	creator	3447	carousel	Food
8	IG0000009	11	brand	9044	reel	Beauty
14	IG0000015	6	brand	10034	reel	Lifestyle
20	IG0000021	10	creator	4542	image	Music

5 rows × 23 columns

```

: #sort by follower count
print("\n===== SORT BY FOLLOWER COUNT (DESCENDING) =====")
display(df.sort_values(by="follower_count", ascending=False).head())

```

===== SORT BY FOLLOWER COUNT (DESCENDING) =====

	post_id	account_id	account_type	follower_count	media_type	content_category
26207	IG0026208	20	brand	31095	image	Fashion
18029	IG0018030	20	brand	31095	image	Comedy
21849	IG0021850	20	creator	31095	reel	Travel
13010	IG0013011	20	brand	31095	image	Photography
1285	IG0001286	20	brand	31095	reel	Fashion

5 rows x 7 columns

```

#print follower count statistics
print("\n===== FOLLOWER COUNT STATISTICS =====")

print("Mean :", df["follower_count"].mean())

print("Median :", df["follower_count"].median())

print("Mode :")
print(df["follower_count"].mode())

print("Standard Deviation :", df["follower_count"].std())

```

===== FOLLOWER COUNT STATISTICS =====

Mean : 10278.305476849227

Median : 9044.0

Mode :

0 5824

1 8167

Name: follower_count, dtype: int64

Standard Deviation : 6691.862669463024

```
#print descriptive statistics
print("\n===== DESCRIPTIVE STATISTICS =====")
display(df.describe())
```

===== DESCRIPTIVE STATISTICS =====

	account_id	follower_count	has_call_to_action	post_hour	likes
count	29999.000000	29999.000000	29999.000000	29999.000000	29999.000000
mean	10.567819	10278.305477	0.348778	11.499417	287.653588
std	5.763952	6691.862669	0.476592	6.900587	317.647682
min	1.000000	3083.000000	0.000000	0.000000	0.000000
25%	6.000000	5824.000000	0.000000	6.000000	104.000000
50%	11.000000	9044.000000	0.000000	12.000000	199.000000
75%	16.000000	10739.000000	1.000000	17.000000	363.000000
max	20.000000	31095.000000	1.000000	23.000000	10632.000000

```
#print correlation matrix
print("\n===== CORRELATION MATRIX =====")

corr = df[
    [
        "follower_count",
        "comments",
        "shares",
        "saves",
        "reach",
        "impressions",
        "engagement_rate",
        "followers_gained",
        "caption_length",
        "hashtags_count"
    ]
].corr()

display(corr)
```

===== CORRELATION MATRIX =====

	follower_count	comments	shares	saves	reach	impressions	engagement_rate	followers_gained	caption_length	hashtags_count
follower_count	1.000000	-0.002921	-0.004888	-0.002551	0.002262	0.001323	-0.005620	0.012654	0.000401	0.006265
comments	-0.002921	1.000000	0.886264	0.886372	0.690038	0.674126	0.451598	-0.005703	0.002478	0.004257
shares	-0.004888	0.886264	1.000000	0.920927	0.715808	0.699672	0.468681	-0.002571	0.004931	0.006597
saves	-0.002551	0.886372	0.920927	1.000000	0.722138	0.706507	0.478121	-0.003144	0.000721	0.009264
reach	0.002262	0.690038	0.715808	0.722138	1.000000	0.985520	0.006187	-0.004840	0.003414	0.002207
impressions	0.001323	0.674126	0.699672	0.706507	0.985520	1.000000	-0.020682	-0.006178	0.003083	0.001800
engagement_rate	-0.005620	0.451598	0.468681	0.478121	0.006187	-0.020682	1.000000	0.000509	0.000497	-0.001246
followers_gained	0.012654	-0.005703	-0.002571	-0.003144	-0.004840	-0.006178	0.000509	1.000000	-0.003091	-0.003561
caption_length	0.000401	0.002478	0.004931	0.000721	0.003414	0.003083	0.000497	-0.003091	1.000000	0.000321
hashtags_count	0.006265	0.004257	0.006597	0.009264	0.002207	0.001800	-0.001246	-0.003561	0.000321	1.000000

```

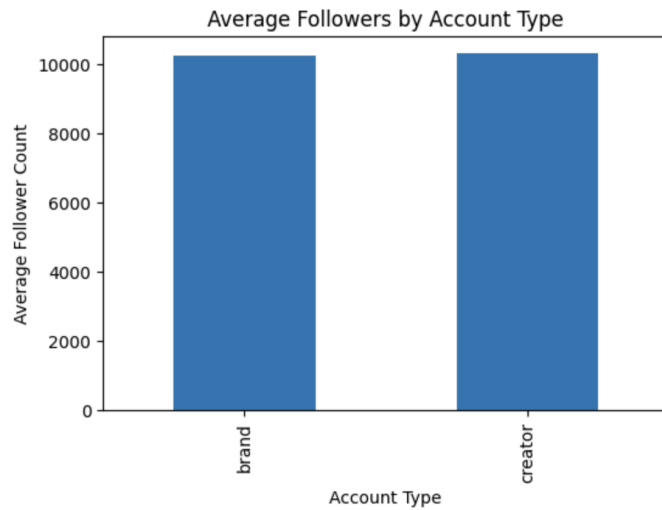
: #display barchar average followers by account type
plt.figure(figsize=(6,4))

account_avg.plot(kind="bar")

plt.title("Average Followers by Account Type")
plt.xlabel("Account Type")
plt.ylabel("Average Follower Count")

plt.show()

```



```

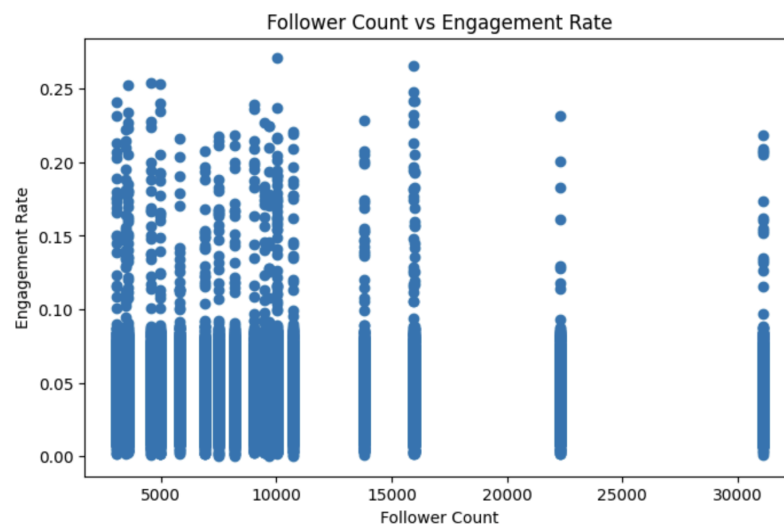
#display scatter plot between engagement rate and follower count
plt.figure(figsize=(8,5))

plt.scatter(
    df["follower_count"],
    df["engagement_rate"]
)

plt.title("Follower Count vs Engagement Rate")
plt.xlabel("Follower Count")
plt.ylabel("Engagement Rate")

plt.show()

```

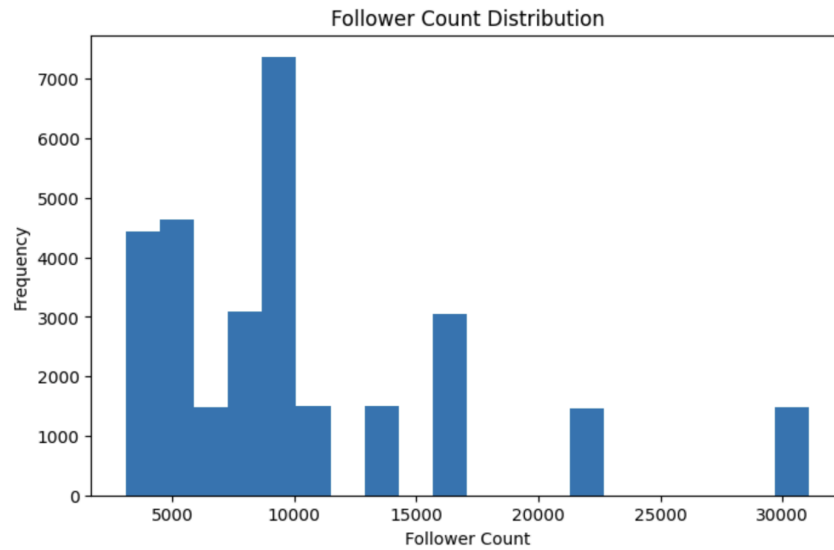


```
: #display histogram for follower count
plt.figure(figsize=(8,5))

plt.hist(df["follower_count"], bins=20)

plt.title("Follower Count Distribution")
plt.xlabel("Follower Count")
plt.ylabel("Frequency")

plt.show()
```



```
#display histogram for engagement rate distirbution
plt.figure(figsize=(8,5))

plt.hist(df["engagement_rate"], bins=20)

plt.title("Engagement Rate Distribution")
plt.xlabel("Engagement Rate")
plt.ylabel("Frequency")

plt.show()
```

